

Pseudojęzyk

1. Instrukcje warunkowe - podstawy

Rozdzielenia algorytmu dokonujemy z użyciem instrukcji `JEŻELI` (można także użyć `JEŻELI, IŻ` (dla używających angielskiego)).

Za instrukcją powinien znajdować się zapis warunkowy, który daje odpowiedź na pytanie wyłącznie: *prawda* (**TAK**) lub *fałsz* (**NIE**). Ponieważ warunki to są technicznie skoki w kodzie, to powinny być na końcu linii operatory dwukropka `:"`.

Przykład filozofii zapisu poprawnego warunku:

„**czy** A jest równe B”, „**czy** wybrany kolor jest zielony”.

Poniżej sprawdzamy, czy dwie liczby są równe. Możemy jednoznacznie odpowiedzieć na to pytanie **TAK** (są sobie równe) lub **NIE** (nie są takie same):

`JEŻELI a = b :`

Przykład filozofii zapisu złego warunku:

„**o ile** A jest większe od B”, „**jakiego koloru** jest ten wybrany”. Poniżej obliczamy wynik z dzielenia dwóch liczb. Jak taki wynik (liczba) ma dać odpowiedź na pytanie TAK lub NIE?

`JEŻELI a / b :`

Zakazane jest tworzenie warunków-zapytań, które choć dają odpowiedź **tak** lub **nie**, są zbyt ogólne w rozumieniu, lub same są pewnym problemem, który wymaga wcześniejszego rozwiązania.

Przykład złego warunku (szukamy najmniejszej liczby spośród kilku liczb): `JEŻELI a jest najmniejsze :`

Powyższy warunek jest niedozwolony, bo w algorytmie wyszukującym najmniejszą liczbę posługujemy się pojęciem „najmniejsza” chociaż nie mamy żadnego mechanizmu, który to sprawdzi. A na jakiej podstawie sprawdzimy, że liczba jest najmniejsza? Jakich wzorów użyjemy? Jak się ta liczba ma do innych spośród których wyszukujemy tej właściwej? No właśnie.

Pseudojęzyk

2. Operatory

Aby uzyskać w warunku odpowiedź *prawda* (**TAK**) lub *fałsz* (**NIE**) musimy użyć zastosować właściwe operatory:

= - „równa się”. Sprawdzamy czy dwa elementy są takie same. **Uwaga!**

Przyjęło się, że w pseudokodzie operatorem porównania jest "=" (operator przypisania ":=" jest używany w obliczeniach). Nie należy mylić tych dwóch operatorów.

!= - „są różne”. Sprawdzamy czy dwa elementy są od siebie odmienne. Można także użyć operatora „**≠**” ale ponieważ nie występuje we wszystkich czcionkach, to „**!=**” wydaje się lepszym rozwiązaniem.

< **>** - operatory „mniejszy” oraz „większy”.

<= **>=** - operatory „mniejszy lub równy” oraz „większy lub równy”. Operator „**=**” zawsze musi być po prawej stronie znaku „**<**” lub „**>**”!

Dobra praktyka programistyczna.

Ponieważ elementy mniejsze są zazwyczaj przedstawiane na osi liczbowej po lewej stronie, to przy używaniu operatorów mniejsza/większa zaleca się zapisywanie warunku właśnie w taki sposób.

~~JEŻELI~~ $a < b$:

Poniższy warunek jest identyczny z tym wyżej. Jest poprawny, ale jego ułożenie może optycznie mylić co do właściwego zrozumienia intencji większa/mniejsza, tym samym łatwiej o pomyłkę w analizie kodu: ~~JEŻELI~~ $a > b$:

Pseudojęzyk

3. Formatowanie kodu

Aby rozdzielić algorytm na dwa różne kierunki nie możemy pisać kodu po prawej i lewej stronie kartki. Zamiast tego, piszemy kod „we wcięciach”, kolejno jeden pod drugim.

Wszystkie instrukcje realizowane w opcji **TAK** zapisujemy bezpośrednio pod **JEŻELI**. Wszystkie instrukcje realizowane w gałęzi **NIE** zapisujemy bezpośrednio pod **WPP** („w przeciwnym przypadku”, ale można użyć podobnego sformułowania np.: **WPW** – „w przeciwnym wypadku”, **JEŻELI_NIE**, czy podobne sugerujące „drugą opcję”).

Kod należący do danej gałęzi powinien być wytabowany względem **JEŻELI/WPP**, któremu podlega.

Od miejsca, gdzie gałęzie **TAK/NIE** danego warunku łączą się ze sobą, zapis powinien być tak samo wytabowany, jak **JEŻELI**, które algorytm rozdzieliło.

Algorytm porównujący dwie liczby:

$a := 5$

$b := 10$

JEŻELI $a < b$:

DRUKUJ „Liczba pierwsza jest mniejsza”

WPP:

DRUKUJ „Liczba druga jest mniejsza”

DRUKUJ „To się drukuje niezależnie co było wcześniej”

Jeśli w którejś z opcji wyboru nie ma żadnych operacji, to możemy ją pominąć w zapisie pseudojęzyka.

Ostatecznie możemy tam wstawić „pustą operację” np. **RÓB_NIC**. Jeśli np. gałąź **fałsz/NIE** jest pusta, to ją pomijamy.

Przykłady obliczania wartości bezwzględnej z liczby:

JEŻELI $a < 0$:

$a := -a$

JEŻELI $a < 0$:

$a := a * -1$

WPP:

$a := a$

JEŻELI $0 \leq a$:

RÓB_NIC

WPP:

$a := 0 - a$

Pseudojęzyk

4. Grupowanie warunków – budowa warunków złożonych

Każdy kolejny warunek podległy innemu warunkowi musimy wytabować (wciąć) stosownie do bloku kodu w jakim się ma znaleźć.

Jeśli kilka warunków występuje po sobie, możemy próbować łączyć je w jeden większy. Wymagane jest jednak, aby takie złożone warunki zapisywać z użyciem **operatorów łączących**.

I (ang. **AND**) - oznacza, że **warunki po obu stronach muszą być prawdziwe**, aby całość była prawdziwa.

LUB (ang. **OR**) – oznacza, że **wystarczy żeby choć jeden z warunków po którejś ze stron był prawdziwy**, aby całość była prawdziwa. **ALBO** (ang. **XOR**) – oznacza, że **tylko jeden warunek po którejś stronie musi być prawdziwy** (prawdziwość obydwu warunków jednocześnie jest traktowana jako całościowa nieprawda).

W przypadku konstruowania warunku bardzo złożonego, z dużej liczby warunków pomniejszych (można więcej niż tylko dwóch), zalecane jest używanie nawiasów przy ich zapisie, aby łatwiej się było połapać o co chodzi.

Przykład sprawdzenia, czy liczba jest z przedziału od 2 do

4: JEŻELI $2 \leq a$:

JEŻELI $a \leq 4$:

DRUKUJ „Siczba w przedziale!”

WPP:

DRUKUJ „Siczba poza przedziałem”

WPP:

DRUKUJ „Siczba poza przedziałem”

... ale możemy zredukować do zapisu:

JEŻELI $(2 \leq a) \text{ I } (a \leq 4)$:

DRUKUJ „Siczba w przedziale!”

WPP:

DRUKUJ „Siczba poza przedziałem”

Nie należy zapisywać warunków złożonych jako np.:

JEŻELI $A < B < C$:

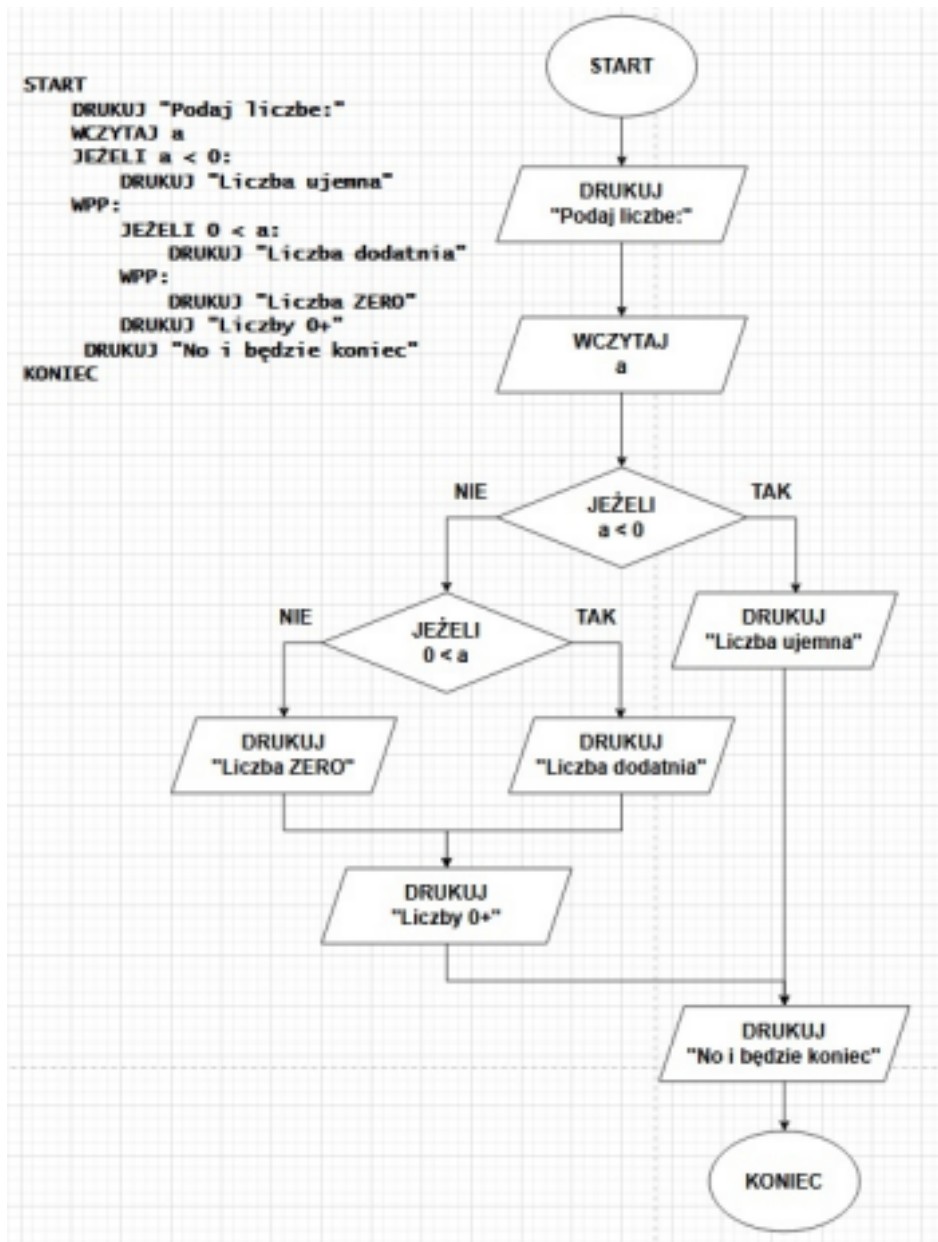
Zamiast tego należy taki warunek rozdzielić na 2 mniejsze zapisane: **JEŻELI ($A < B$) I ($B < C$) :**

Ten pierwszy warunek ma sens wyłącznie matematyczny.

Komputer nie jest w stanie zrozumieć pierwszego zapisu. Procesor jest w stanie zrealizować naraz jedną operację na maksymalnie dwóch zmiennych (mówiąc w wielkim uproszczeniu). Przyzwyczajajmy się myśleć jak prawdziwi programiści. ;)

Pseudojęzyk

5. Przykładowy algorytm – wybór z 3 opcji



Pseudojęzyk

6. Przykładowy algorytm – sprawdzenie parzystości

